HDOC Day-60

Checking Whether Two Strings Are Anagrams Using Functions and Pointers in C

Aim

To write a C program using strings, pointers, user-defined functions, arrays, and character-frequency counting to determine whether two lowercase strings are anagrams. Display "Anagram" when character frequencies match and "Not Anagram" otherwise.

Algorithm

1. Start the program and declare two character arrays s and t.
2. Read both lowercase strings and pass their addresses to check_anagram().
3. Calculate the string lengths using strlen().
4. If lengths differ, print "Not Anagram" and return.
5. Initialize a frequency array of 26 integers to zero.
6. For each character in s, increment its corresponding frequency.
7. For each character in t, decrement its corresponding frequency.
8. If any frequency is nonzero, print "Not Anagram"; otherwise print "Anagram".
9. Stop the program.

C Program

```c
#include <stdio.h>
#include <string.h>

#define MAX 200

void check_anagram(char *s, char *t);

int main()
{
    char s[MAX], t[MAX];

    printf("Enter first string: ");
    scanf("%199s", s);

    printf("Enter second string: ");
    scanf("%199s", t);

    check_anagram(s, t);

    return 0;
}

/* Function to check whether two strings are anagrams */
void check_anagram(char *s, char *t)
{
    int freq[26] = {0};
    int i;
    int len1, len2;

    len1 = strlen(s);
    len2 = strlen(t);

    /* Check whether lengths are equal */
    if (len1 != len2)
    {
        printf("Not Anagram\n");
        return;
    }

    /* Count frequency of characters in first string */
    for (i = 0; i < len1; i++)
    {
        freq[*(s + i) - 'a']++;
    }

    /* Subtract frequency of characters in second string */
    for (i = 0; i < len2; i++)
    {
        freq[*(t + i) - 'a']--;
    }

    /* Check whether all frequencies are zero */
    for (i = 0; i < 26; i++)
    {
        if (freq[i] != 0)
        {
            printf("Not Anagram\n");
            return;
        }
    }

    printf("Anagram\n");
}
```

Explanation

The function check_anagram(char *s, char *t) receives pointers to the first characters of the two strings. Pointer expressions *(s + i) and *(t + i) access the characters at index i. A 26-element frequency array maps a through z to indices 0 through 25. The first string increments counts and the second decrements them. All final counts must equal zero for the strings to be anagrams.

Example: listen and silent

Character	In listen	In silent
e	1	1
i	1	1
l	1	1
n	1	1
s	1	1
t	1	1

Since each letter occurs with the same frequency in both strings, the output is "Anagram". For "hello" and "world", the frequencies differ, so the output is "Not Anagram".

Test Case Table

Test	First String	Second String	Expected Result
1	listen	silent	Anagram
2	anagram	nagaram	Anagram
3	rat	car	Not Anagram
4	hello	world	Not Anagram
5	abc	bca	Anagram
6	aabbcc	abcabc	Anagram
7	aabb	abbb	Not Anagram
8	abcd	abc	Not Anagram
9	a	a	Anagram
10	a	b	Not Anagram

Sample Outputs

Valid Anagram

```
Enter first string: listen
Enter second string: silent
Anagram
```

Another Valid Anagram

```
Enter first string: anagram
Enter second string: nagaram
Anagram
```

Not an Anagram

```
Enter first string: hello
Enter second string: world
Not Anagram
```

Different String Lengths

```
Enter first string: abcd
Enter second string: abc
Not Anagram
```

Repeated Characters

```
Enter first string: aabbcc
Enter second string: abcabc
Anagram
```

Result

The C program was designed using strings, pointers, user-defined functions, arrays, loops, and conditional statements. The check_anagram() function compares character frequencies regardless of order and correctly prints "Anagram" or "Not Anagram" for the given lowercase inputs.